

ASSIGNMENT-11.3

2303A51595

B-10

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Prompt : Create a simple Java program for a Contact Manager using ArrayList. Each contact should store name and phone number. Add functions to add contact, search contact, and delete contact. Keep the code simple for students.

Code :

```
class Node {
    String name;
    String phone;
    Node next;

    Node(String name, String phone) {
        this.name = name;
        this.phone = phone;
        this.next = null;
    }
}

public class ContactManagerLinkedList {

    Node head = null;

    void addContact(String name, String phone) {
        Node newNode = new Node(name, phone);
        newNode.next = head;
        head = newNode;
        System.out.println("Contact Added");
    }

    void searchContact(String name) {
        Node temp = head;

        while (temp != null) {
            if (temp.name.equals(name)) {
                System.out.println("Found: " + temp.name + " " + temp.phone);
                return;
            }
        }
    }
}
```

```

    }
    temp = temp.next;
}

System.out.println("Contact not found");
}

void deleteContact(String name) {
    Node temp = head;
    Node prev = null;
    while (temp != null) {
        if (temp.name.equals(name)) {

            if (prev == null)
                head = temp.next;
            else
                prev.next = temp.next;

            System.out.println("Contact deleted");
            return;
        }
        prev = temp;
        temp = temp.next;
    }
    System.out.println("Contact not found");
}

public static void main(String[] args) {

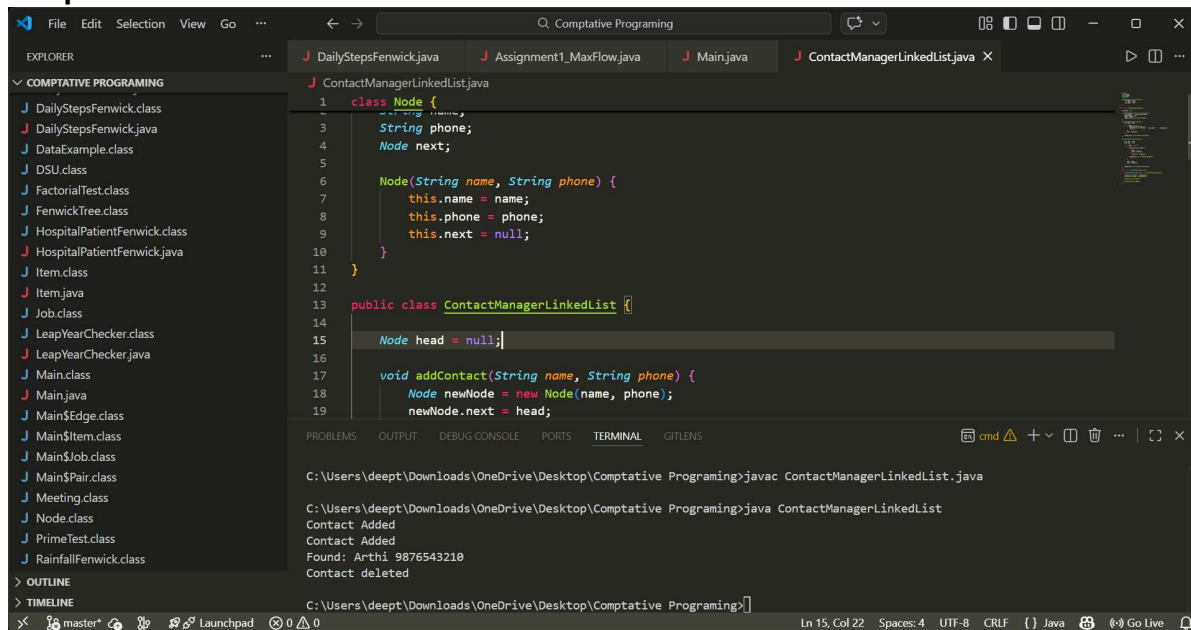
    ContactManagerLinkedList cm = new ContactManagerLinkedList();

    cm.addContact("Arthi", "9876543210");
    cm.addContact("xsa", "9876543333");

    cm.searchContact("Arthi");
    cm.deleteContact("Xyzz");
}
}

```

Output :



```
1 class Node {
2     String name;
3     String phone;
4     Node next;
5
6     Node(String name, String phone) {
7         this.name = name;
8         this.phone = phone;
9         this.next = null;
10    }
11 }
12
13 public class ContactManagerLinkedList {
14
15     Node head = null;
16
17     void addContact(String name, String phone) {
18         Node newNode = new Node(name, phone);
19         newNode.next = head;
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL GIT LENS

```
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>javac ContactManagerLinkedList.java
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>java ContactManagerLinkedList
Contact Added
Contact Added
Found: Arthi 9876543210
Contact deleted
```

Explanation :

This program implements a Contact Manager using ArrayList.

Each contact stores name and phone number.

Operations:

- Add Contact
- Search Contact
- Delete Contact

ArrayList is easy to use but deletion may take more time because elements shift.

Task 2: Library Book Search System (Queue & Priority Queue)

Prompt : Create a simple Java program using Queue for library book requests. Students and faculty request books. Also create a Priority Queue where faculty requests are processed first.

Code:

Queue Implementation

Code

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
public class LibraryQueue {
```

```
    public static void main(String[] args) {
```

```
        Queue<String> queue = new LinkedList<>();
```

```

        queue.add("Student Request");
        queue.add("Faculty Request");
        queue.add("Student Request");

        System.out.println("Processing: " + queue.remove());
        System.out.println("Processing: " + queue.remove());
    }
}

```

Priority Queue Implementation :

Prompt

Create a Java program using PriorityQueue.

Faculty requests should have higher priority than student requests.

Code :

```

import java.util.PriorityQueue;
class Request implements Comparable<Request> {

    int priority;
    String name;

    Request(int priority, String name) {
        this.priority = priority;
        this.name = name;
    }

    public int compareTo(Request r) {
        return this.priority - r.priority;
    }
}

public class LibraryPriorityQueue {

    public static void main(String[] args) {

        PriorityQueue<Request> pq = new PriorityQueue<>();

        pq.add(new Request(2, "Student Request"));
        pq.add(new Request(1, "Faculty Request"));
        pq.add(new Request(2, "Student Request"));
    }
}

```

```

        while (!pq.isEmpty()) {
            System.out.println("Processing: " + pq.poll().name);
        }
    }
}

```

Output :

```

// LibraryPriorityQueue.java
import java.util.PriorityQueue;

class Request implements Comparable<Request> {
    int priority;
    String name;

    Request(int priority, String name) {
        this.priority = priority;
        this.name = name;
    }

    public int compareTo(Request r) {
        return this.priority - r.priority;
    }
}

public class LibraryPriorityQueue {
    public static void main(String[] args) {
        // ...
    }
}

```

Terminal Output:

```

C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>javac LibraryPriorityQueue.java
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>java LibraryPriorityQueue
Processing: Faculty Request
Processing: Student Request
Processing: Student Request

```

```

// LibraryPriorityQueue.java
import java.util.PriorityQueue;

class Request implements Comparable<Request> {
    int priority;
    String name;

    Request(int priority, String name) {
        this.priority = priority;
        this.name = name;
    }

    public int compareTo(Request r) {
        return this.priority - r.priority;
    }
}

public class LibraryPriorityQueue {
    public static void main(String[] args) {
        // ...
    }
}

```

Terminal Output:

```

C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>javac LibraryPriorityQueue.java
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>java LibraryPriorityQueue
Processing: Faculty Request
Processing: Student Request
Processing: Student Request

```

Explanation :

Queue follows FIFO (First In First Out).

The request that comes first is processed first.

Task 3: Emergency Help Desk (Stack)**Prompt :**

Create a simple Java program using Stack. Include push, pop, peek operations. Simulate five support tickets.

Code :

```
import java.util.Scanner;
import java.util.Stack;

public class HelpDeskStack {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Stack<String> stack = new Stack<>();

        System.out.print("Enter number of tickets: ");
        int n = sc.nextInt();
        sc.nextLine();

        for(int i=1;i<=n;i++){
            System.out.print("Enter ticket "+i+": ");
            String ticket = sc.nextLine();
            stack.push(ticket);
        }
        System.out.println("Top Ticket: " + stack.peek());

        System.out.println("Resolving Tickets (LIFO):");
        while(!stack.isEmpty()){
            System.out.println("Resolved: " + stack.pop());
        }

        sc.close();
    }
}
```

Output :

```
File Edit Selection View Go ...
Q Comptative Programing

EXPLORER
COMPTATIVE PROGRAMING
  ass1.class
  ass1.java
  Assignment1_MaxFlow.class
  Assignment1_MaxFlow.java
  Calculator.class
  ContactManagerLinkedList.class
  DailySalesFenwick.class
  DailySalesFenwick.java
  DailyStepsFenwick.class
  DailyStepsFenwick.java
  DataExample.class
  DSU.class
  FactorialTest.class
  FenwickTree.class
  HelpDeskStack.class
  HelpDeskStack.java
  HospitalPatientFenwick.class
  HospitalPatientFenwick.java
  Item.class
  Item.java
  Job.class
  LeapYearChecker.class
  LeapYearChecker.java
  ...
OUTLINE
TIMELINE
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>

HelpDeskStack.java
1 import java.util.Scanner;
2 import java.util.Stack;
3
4 public class HelpDeskStack {
5
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9         Stack<String> stack = new Stack<>();
10
11         System.out.print("Enter number of tickets: ");
12         int n = sc.nextInt();
13         sc.nextLine();
14
15         for(int i=1;i<=n;i++){
16             System.out.print("Enter ticket "+i+": ");
17             String ticket = sc.nextLine();
18             stack.push(ticket);
19         }
20     }
21 }
```

```
Problems OUTPUT DEBUG CONSOLE PORTS TERMINAL GIT LENS
cmd
Enter number of tickets: 2
Enter ticket 1: 1
Enter ticket 2: 2
Top Ticket: 2
Resolving Tickets (LIFO):
Resolved: 2
Resolved: 1

Ln 4, Col 14 (13 selected) Spaces: 4 UTF-8 CRLF {} Java Go Live
```

Explanation :

This program implements a **Queue** for cafeteria orders.

Operations used:

- **add()** → place order
- **remove()** → serve order

Queue follows **FIFO (First In First Out)** principle.

So the **first order placed will be served first**.

Task 4: Hash Table (User Input)

Prompt :

Create a simple Java HashTable program with insert, search and delete methods.

Use user input to enter keys.

Handle collisions using chaining.

Code :

```
import java.util.*;
```

```
class HashTable {
```

```
    int size = 10;
```

```
    LinkedList<String>[] table;
```

```
    HashTable() {
```

```
        table = new LinkedList[size];
```

```
        for(int i=0;i<size;i++)
```

```

        table[i] = new LinkedList<>();
    }

    int hashFunction(String key){
        return Math.abs(key.hashCode()) % size;
    }

    void insert(String key){
        int index = hashFunction(key);
        table[index].add(key);
        System.out.println("Inserted: "+key);
    }

    void search(String key){
        int index = hashFunction(key);

        if(table[index].contains(key))
            System.out.println("Found: "+key);
        else
            System.out.println("Not Found");
    }

    void delete(String key){
        int index = hashFunction(key);

        if(table[index].remove(key))
            System.out.println("Deleted: "+key);
        else
            System.out.println("Key not found");
    }
}

public class HashTableDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        HashTable ht = new HashTable();

        System.out.print("Enter number of elements: ");
    }
}

```



```

int n = sc.nextInt();
sc.nextLine();

for(int i=1;i<=n;i++){
    System.out.print("Enter key "+i+": ");
    String key = sc.nextLine();
    ht.insert(key);
}

System.out.print("Enter key to search: ");
String searchKey = sc.nextLine();
ht.search(searchKey);

System.out.print("Enter key to delete: ");
String deleteKey = sc.nextLine();
ht.delete(deleteKey);

sc.close();
}
}

```

Output :

The screenshot shows an IDE with the following components:

- EXPLORER:** A list of files under 'COMPTATIVE PROGRAMING'. The file 'HashTableDemo.java' is selected.
- EDITOR:** Displays the code for 'HashTableDemo.java'. The visible code includes:


```

class HashTable {
    ...

    void delete(String key){
        int index = hashFunction(key);
        if(table[index].remove(key))
            System.out.println("Deleted: "+key);
    }
}

```
- TERMINAL:** Shows the execution of the program. The output is:


```

Note: Recompile with -Xlint:unchecked for details.

C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programming>java HashTableDemo
Enter number of elements: 3
Enter key 1: 1
Inserted: 1
Enter key 2: 2
Inserted: 2
Enter key 3: 3
Inserted: 3
Enter key to search: 4
Not Found
Enter key to delete: 2
Deleted: 2

C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programming>

```

Explanation :

This program implements a **Hash Table** using **chaining for collision handling**.

Operations:

- **Insert** add key into hash table
- **Search** find key
- **Delete** remove key

The **hash function converts a key into an index**, where data is stored.

If two keys map to the same index, **collision occurs**, and we store them using **LinkedList chaining**.

Task 5: Cafeteria Order Queue (User Input)

Prompt :

Create a Java program for Cafeteria Order System using Queue. Take orders from user input and process them using FIFO principle.

Code :

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class CafeteriaQueue {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Queue<String> orders = new LinkedList<>();

        System.out.print("Enter number of orders: ");
        int n = sc.nextInt();
        sc.nextLine();

        for(int i=1;i<=n;i++){
            System.out.print("Enter order "+i+": ");
            String order = sc.nextLine();
            orders.add(order);
        }

        System.out.println("Serving Orders:");

        while(!orders.isEmpty()){
            System.out.println("Serving: "+orders.remove());
        }
    }
}
```

```

        sc.close();
    }
}

```

Output :

The screenshot shows an IDE with the following code in `CafeteriaQueue.java`:

```

1  import java.util.LinkedList;
2  import java.util.Queue;
3  import java.util.Scanner;
4
5  public class CafeteriaQueue {
6
7      public static void main(String[] args) {
8
9          Scanner sc = new Scanner(System.in);
10         Queue<String> orders = new LinkedList<>();
11
12         System.out.print("Enter number of orders: ");
13         int n = sc.nextInt();
14         sc.nextLine();
15
16         for(int i=1;i<=n;i++){

```

The terminal output shows the program's execution:

```

C:\Users\deept\Downloads\OneDrive\Desktop\Comptative Programing>java CafeteriaQueue
Enter number of orders: 3
Enter order 1: soap
Enter order 2: jndsaj
Enter order 3: zxhsa
Serving Orders:
Serving: soap
Serving: jndsaj
Serving: zxhsa

```

Explanation :

This program implements a **Queue** for cafeteria orders.

Operations used:

- **add()** place order
- **remove()** serve order

Queue follows **FIFO (First In First Out)** principle.

So the **first order placed will be served first**.